



DQN funcionamiento

👤 Creada por	 Roberto Rodríguez Núñez
🕒 Hora de creación	@18 de noviembre de 2025 14:32
☰ Categoría	Documentación técnica
👤 Última edición por	 Roberto Rodríguez Núñez
🕒 Fecha de última actualización	@28 de enero de 2026 20:20

PARA QUE SIRVE LA TARGET NETWORK

Tienes toda la razón. Es la paradoja que más cuesta entender: **"Si la Target Network es igual de tonta al principio, ¿cómo ayuda a corregir a la otra?"**.

La respuesta corta es: **Porque la REALIDAD (la Recompensa inmediata) pesa más que la opinión tonta de la Target Network.**

Vamos a ver matemáticamente cómo la Target Network "tonta" participa en el cálculo y por qué, aun así, el sistema funciona.

Escenario: "Dos Gemelos Tontos"

- **Estado Actual (\$\$\$):** Precio 10 (Caro).
- **Estado Futuro (\$S'\$):** Precio 2 (Barato).
- **Acción:** Cargar Batería.
- **Recompensa (\$R\$):** -100 (El castigo de la realidad).

MOMENTO 0: La clonación

Ambas redes empiezan con pesos aleatorios positivos (creen que cargar siempre es genial).

1. 🟢 **Main Q-Network:** $w = 0.8$
2. 🟡 **Target Network:** $w' = 0.8$ (Igual de tonta).

EL CÁLCULO CRÍTICO (Donde participa la Target)

Queremos calcular el Target Value (\$Y\$). La fórmula es:

$$Y = \text{Recompensa} + (\gamma \times \text{Predicción_De_La_Target_Network})$$

Vamos a sustituir los números. Aquí es donde entra la Target Network con su peso tonto de 0.8 mirando al futuro (Precio 2):

1. Opinión de la Target Network:

$$Q_{\text{target}}(S') = \text{PrecioFuturo} \times w' = 2 \times 0.8 = \mathbf{1.6}$$

(La Target dice: "El futuro pinta bien, ganaremos 1.6 puntos". ¡Es mentira! Pero es lo que cree).

2. Cálculo del Target Value (\$Y\$):

$$Y = -100 + (0.9 \times 1.6) = -100 + 1.44 = \mathbf{-98.56}$$

¡FÍJATE AQUÍ!

Aunque la Target Network dijo una tontería (\$+1.44\$), la Recompensa Real (\$-100\$) es tan fuerte que aplasta la tontería. El resultado final (\$-98.56\$) es muy negativo.

La Target Network "tonta" no arruinó el cálculo porque la experiencia inmediata (el -100) ancló el resultado a la realidad.

EL AJUSTE (Main Network se vuelve lista)

Ahora calculamos el error de la Main Network.

- **Main Network dice:** $10 \times 0.8 = \mathbf{8}$ (Cree que ganará dinero).
- **Target Value (La Verdad Calculada):** $\mathbf{-98.56}$. $Loss = (-98.56 - 8)^2$

La Main Network se da cuenta de que está lejísimos. El gradiente la obliga a cambiar su peso de **0.8** a **-0.5**.

EL RESULTADO: La Divergencia Necesaria

Después de este primer paso, pasa algo curioso:

1.  **Main Q-Network:** $w = -0.5$ (¡Ya es lista! Sabe que precio alto = malo).
2.  **Target Network:** $w' = 0.8$ (Sigue siendo tonta).

¿Por qué no actualizamos la Target ya?

Si actualizáramos la Target ahora mismo a -0.5 , en el siguiente paso la "referencia" cambiaría de golpe.

Al mantener a la Target "tonta" y estable durante unos 1000 pasos, nos aseguramos de que la Main Network tenga tiempo de converger hacia ese

valor de \$-98.56\$ sin que le movamos la portería.

Resumen del proceso:

1. Al principio, la **Recompensa (\$R\$)** es la que enseña (porque domina la fórmula).
2. La Target Network "tonta" solo aporta un pequeño ruido (\$+1.44\$) que es despreciable comparado con el castigo (\$-100\$).
3. Con el tiempo, cuando hagamos el **Update** (copiar pesos), la Target también será lista (\$-0.5\$).
4. Ahí es cuando la Target empezará a hacer predicciones útiles sobre el futuro a largo plazo, más allá de la recompensa inmediata.

COMO FUNCIONA EL DQN

1. Inicialización de las Redes (Paso 0)

Antes de que el agente realice cualquier acción, ambas redes se construyen con una estructura idéntica, pero con valores distintos:

- **Valores Iniciales (\$W\$ y \$W'\$):** Los pesos de ambas redes (\$W\$ para \$Q_{\text{net}}\$ y \$W'\$ para \$Target_{\text{net}}\$) se inicializan con valores **aleatorios pequeños** (típicamente entre 0 y 1 o usando métodos como Xavier).
- **Clonación:** La \$Target_{\text{net}}\$ es una copia directa de la \$Q_{\text{net}}\$:
$$\mathbf{W}' = \mathbf{W}$$
- **El Agente es "Tonto":** Al inicio, ambas \$Q_{\text{net}}\$ y \$Target_{\text{net}}\$ predicen valores erróneos (cerca de cero), ya que los pesos son aleatorios.

2. El Ciclo de Aprendizaje Gradual (Pasos 1 a 1000)

El entrenamiento ocurre en dos bucles entrelazados. La **estabilidad matemática** reside en que la Targetnet mantiene fijo el objetivo durante estos 1000 pasos.

Fase A: El Cálculo del Target (El Objetivo Fijo)

Este cálculo ocurre cuando se procesa un **Batch** (muestras aleatorias del Replay Buffer).

El valor que queremos que la \$Q_{\text{net}}\$ aprenda a predecir es el **Target Value (\$Y_t\$)**, que se calcula con la Ecuación de Bellman:
$$Y_t = R_t + \gamma \max$$

$Q_{\text{TargetNet}}(S_{t+1}, a')$ {\text{Fijo para este ciclo}}

- **R_t (Recompensa):** Es el ancla de la realidad (ej., -10). Es un valor **constante** y no depende de ningún peso.
- **$Q_{\text{TargetNet}}(S_{t+1}, a')$:** Usa los **pesos congelados** W' . Este valor es constante durante los 1000 pasos.

Ejemplo (Continuación del ciclo 1):

Si $R = -10$ y $\max Q_{\text{TargetNet}}$ (con pesos W' aleatorios) es 0:

$$Y_t = -10 + (0.9 \times 0) = \mathbf{-10}$$

Conclusión: El objetivo que la Qnet persigue durante este ciclo está fijado en -10.

Fase B: El Trabajo del Gradiente (Actualización de la Qnet)

La Q_{net} es la única que ajusta sus pesos (W) para reducir la pérdida (Loss) entre su predicción y el Target Fijo Y_t .

1. Cálculo de la Pérdida (Loss): Se usa la diferencia entre la predicción y el objetivo fijo.

$$\text{Loss} = \left(Y_t - Q_{\text{Net}}(S_t, a_t; W) \right)^2$$

2. **Cálculo del Gradiente:** Se calcula la derivada de la Pérdida con respecto a los pesos W . Este valor indica la dirección de máxima subida del error.

3. Actualización de Pesos (W): Los pesos se mueven en la dirección opuesta al gradiente (Descenso de Gradiente), controlado por el Learning Rate (α):

$$\mathbf{W}_{\text{nuevo}} = \mathbf{W}_{\text{viejo}} - \alpha \nabla_{\mathbf{W}} \text{Loss}$$

Lo que ocurre en los 1000 pasos:

- **Inicio:** La Q_{Net} predice 0. Error grande. El $\nabla_{\mathbf{W}} \text{Loss}$ es grande. Los pesos W se mueven mucho.
- **Progreso:** Esto se repite en cientos de *batches* aleatorios. Los pesos W se ajustan gradualmente, acercando la predicción de la Q_{Net} a -10.
- **Fin del Ciclo (Paso 1000):** La Q_{Net} ha convergido, y su predicción es ahora, por ejemplo, -9.95.

3. 🔄 El Salto de Estabilidad (Target Update)

Solo al completar el ciclo (Paso 1000, un **hiperparámetro** que elegimos), la

Targetnet se actualiza. $\mathbf{W}'_{\text{nuevo}} = \mathbf{W}_{\text{actual}}$

- Q_{Net} (La Aprendiz) le da su conocimiento a la $Target_{\text{Net}}$ (La Maestra).

- Ahora la $Target_{\text{Net}}$ predice **-9.95** (mucho más cerca de la realidad que el 0 inicial).

- Esto **reinicia** la estabilidad. La Q_{Net} comienza un nuevo ciclo de 1000 pasos persiguiendo un objetivo **aún más preciso** (que ahora es $-10 + 0.9 \times (-9.95) \approx -18.95$).